

Contents

1	Retrieval-Augmented Generation (RAG)	3
1.1	Why RAG is Needed	3
2	Document Chunking	4
2.1	Chunk Overlap	4
2.2	Types of Chunking	5
2.2.1	Fixed-Size Chunking	5
2.2.2	Sentence-Based Chunking	5
2.2.3	Paragraph-Based Chunking	5
2.2.4	Semantic Chunking	5
3	Embeddings	6
4	Vector Database	6
5	Similarity Search	7
6	Prompt Augmentation	7
7	Complete RAG Workflow	8
7.1	Document Indexing	8
7.2	Question Answering	8
8	Application in AI Scholar	8

““

““

1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an advanced architecture that combines the reasoning capabilities of Large Language Models (LLMs) with external knowledge retrieval systems. Instead of relying solely on the knowledge acquired during pre-training, RAG retrieves relevant information from external documents and supplies it to the language model before generating a response.

The term RAG consists of three major stages:

- **Retrieval:** Search for the most relevant information from an external knowledge source.
- **Augmentation:** Add the retrieved information to the prompt provided to the language model.
- **Generation:** Generate an answer using both the retrieved information and the reasoning capability of the language model.

This architecture enables AI systems to answer questions based on user-provided documents instead of depending only on pre-trained knowledge.

1.1 Why RAG is Needed

Large Language Models possess extensive general knowledge; however, they suffer from several limitations:

- They cannot remember documents uploaded by users unless those documents are included in the prompt.
- Processing an entire document for every query is computationally expensive.
- Long documents often exceed the model’s context window.
- The model may generate hallucinated information when the required knowledge is unavailable.

RAG addresses these limitations by retrieving only the most relevant sections of a document before generating an answer.

The major advantages of RAG include:

- Improved factual accuracy.

- Reduced hallucinations.
- Lower token consumption.
- Faster response generation.
- Ability to answer questions about newly uploaded documents.

2 Document Chunking

Chunking is the process of dividing a large document into smaller and meaningful pieces known as *chunks*. Instead of treating an entire PDF as one large text document, it is divided into multiple smaller segments.

For example, a 300-page textbook can be divided into hundreds of individual chunks, each containing a few paragraphs of related information.

The primary objectives of chunking are:

- Reduce document size for processing.
- Improve information retrieval.
- Preserve semantic meaning.
- Enable efficient vector search.
- Reduce computational cost.

Each chunk generally contains between 300 and 1000 words depending on the selected chunking strategy.

2.1 Chunk Overlap

If documents are divided strictly into fixed-size chunks, important information located near chunk boundaries may be lost.

To overcome this limitation, chunk overlap is introduced. During overlapping, a small portion of one chunk is repeated in the next chunk.

For example,

- Chunk 1: Words 1–500
- Chunk 2: Words 450–950
- Chunk 3: Words 900–1400

The overlapping region preserves contextual continuity between adjacent chunks and significantly improves retrieval accuracy.

2.2 Types of Chunking

Several chunking strategies are commonly employed in Retrieval-Augmented Generation systems.

2.2.1 Fixed-Size Chunking

The document is divided after a predefined number of words or tokens.

Advantages

- Simple implementation.
- Fast processing.

Disadvantages

- May split sentences.
- Context can be partially lost.

2.2.2 Sentence-Based Chunking

Entire sentences are grouped together to form chunks.

Advantages

- Preserves sentence structure.
- Better readability.

2.2.3 Paragraph-Based Chunking

Each paragraph forms an independent chunk.

This approach is particularly effective for books, research papers, and technical documentation.

2.2.4 Semantic Chunking

The document is divided based on topic changes rather than word count.

Semantic chunking generally produces the highest retrieval accuracy because each chunk represents a complete concept.

3 Embeddings

After chunking, each chunk is transformed into a numerical vector representation known as an embedding.

An embedding captures the semantic meaning of a text instead of merely representing individual words.

For example, the following sentences produce similar embeddings:

- Machine Learning is a branch of Artificial Intelligence.
- Artificial Intelligence includes Machine Learning techniques.

Although the wording differs, both sentences possess similar meanings, causing their embedding vectors to be located close together within the vector space.

Embedding models commonly used include:

- OpenAI Embedding Models
- Gemini Embeddings
- Sentence Transformers
- BGE Embeddings
- E5 Embeddings

4 Vector Database

After embeddings are generated, they are stored inside a vector database.

Unlike conventional databases that search using exact keywords, vector databases search using semantic similarity.

Popular vector databases include:

- pgvector
- Pinecone
- Weaviate
- ChromaDB
- Milvus
- Qdrant

For the AI Scholar project, embeddings are stored using the pgvector extension within PostgreSQL.

Each record generally contains:

- Document Identifier
- Chunk Identifier
- Original Chunk Text
- Metadata
- Embedding Vector

5 Similarity Search

When a user submits a question, the question itself is converted into an embedding vector using the same embedding model employed during document indexing.

The vector database computes similarity scores between the question embedding and all stored chunk embeddings.

The chunks with the highest similarity scores are retrieved.

Similarity search commonly uses:

- Cosine Similarity
- Euclidean Distance
- Dot Product Similarity

Typically, only the top three to five most relevant chunks are retrieved before generating the final response.

6 Prompt Augmentation

After retrieval, the selected chunks are inserted into the prompt provided to the Large Language Model.

The prompt generally consists of:

- System Instructions
- Retrieved Context
- User Question

By incorporating relevant document content into the prompt, the language model generates responses that are grounded in the uploaded document rather than relying solely on pre-trained knowledge.

This process is referred to as prompt augmentation.

7 Complete RAG Workflow

The complete Retrieval-Augmented Generation pipeline consists of two major phases.

7.1 Document Indexing

1. Upload PDF.
2. Extract document text.
3. Divide the text into chunks.
4. Generate embeddings for every chunk.
5. Store embeddings inside the vector database.

7.2 Question Answering

1. Receive the user's question.
2. Generate an embedding for the question.
3. Search the vector database.
4. Retrieve the most relevant chunks.
5. Augment the prompt with retrieved chunks.
6. Send the prompt to the Large Language Model.
7. Generate the final response.

8 Application in AI Scholar

Within the AI Scholar platform, Retrieval-Augmented Generation is used to provide intelligent question-answering over uploaded academic documents.

The implementation workflow is as follows:

1. Student uploads a PDF.

2. The PDF is converted into plain text.
3. The extracted text is divided into semantic chunks.
4. Embeddings are generated for each chunk.
5. Embeddings are stored using pgvector within PostgreSQL.
6. When a question is asked, the question embedding is generated.
7. Similarity search retrieves the most relevant chunks.
8. Retrieved chunks are added to the prompt.
9. The Large Language Model generates a context-aware response.

This architecture ensures accurate, scalable, and efficient document understanding while minimizing hallucinations and reducing computational cost.